

Arquitectura

Proyecto: LavadoCarros

Version: 1.0

Fecha: 15/09/2026

Elaborado por: Auditoria tecnica

Resumen

Tres contenedores: **frontend React servido por nginx, API Express y PostgreSQL**. Sin cache ni colas. Integraciones externas de correo (SMTP) y mensajería (Twilio WhatsApp y SMS).

Capa	Tecnología	Version	Fuente
Frontend	React + TypeScript + Tailwind	React 19.2.4, TS 4.9.5	frontend/package.json
Servidor web	nginx (imagen del build del frontend)	—	frontend/Dockerfile , frontend/nginx.conf
Backend	Node.js + Express	Express 4.18.2	backend/package.json
ORM	Prisma	5.10.0	backend/package.json
Base de datos	PostgreSQL	15.18 (verificado en ejecución)	SELECT version()
Autenticación	JWT (jsonwebtoken) + bcryptjs	9.0.2 / 2.4.3	backend/package.json
Subida de archivos	multer	1.4.5-lts.1	backend/package.json
Documentación API	Swagger UI	—	backend/src/index.js
Correo	SMTP	—	variables SMTP_*
Mensajería	Twilio (WhatsApp + SMS)	—	variables TWILIO_*

Diagrama de arquitectura

```
flowchart TB
    U["Cliente / Operador /<br/>Lavador / Admin"]
    subgraph HOST["Servidor 192.168.3.124"]
        FE["frontend<br/>nginx + React<br/>:3041"]
        BE["backend<br/>Express + Prisma<br/>:3042"]
        DB["postgres:15<br/>:5437"]
        VOL[["uploads_data<br/>evidencias y comprobantes"]]
    end
    SMTP["Servidor SMTP"]
    TW["Twilio<br/>WhatsApp / SMS"]

    U -->|HTTP| FE
    U -->|"HTTP /api"| BE
    BE --> DB
    BE --> VOL
    BE -->|correo| SMTP
    BE -->|mensajes| TW
```

Nota: el puerto 3042 del backend **esta publicado directamente**, así que el navegador puede llamarlo sin pasar por el frontend. El propio código lo reconoce: el comentario de `rateLimit.middleware.js` dice *"la API vive detras de nginx y ademas publica el puerto 3042, así que la IP se puede falsear"*.

Diagrama de componentes del backend

```
flowchart LR
    R["routes/<br>13 archivos"] --> C["controllers/"]
    C --> S["services/"]
    S --> P["Prisma Client"]
    P --> DB["PostgreSQL"]
    M1["middleware/<br>auth"] -.-> R
    M2["middleware/<br>rateLimit"] -.-> R
    M3["middleware/<br>errorHandler"] -.-> R
    CFG["config/<br>upload.js"] -.-> C
```

Separación en capas correcta: rutas → controladores → servicios → Prisma, con middleware transversal.

Flujo de datos: reserva y pago con comprobante

```
sequenceDiagram
    participant C as Cliente
    participant A as API
    participant D as PostgreSQL
    participant O as Operador

    C->>A: GET /api/reservas/disponibilidad
    A->>D: consulta plazas y agenda
    A-->>C: franjas libres
    C->>A: POST /api/reservas
    A->>D: crea Reserva + HistorialReserva
    O->>A: PUT /api/agenda/asignaciones/:id/lavador
    A->>D: AsignacionAgenda
    Note over A,D: el lavador ejecuta y sube evidencia
    C->>A: POST /api/pagos/:id/pagos/comprobante
    A->>A: multer valida tipo y tamaño
    A->>D: Pago en estado "por validar"
    O->>A: valida el comprobante
    A->>D: Pago confirmado + HistorialReserva
```

Decisiones de diseño observadas

La autenticación revalida contra la base de datos en cada petición. El comentario del propio código lo explica: *"El token dura 24h, así que sin esta"*

consulta un usuario suspendido seguiria operando hasta un dia entero". Es una decision correcta y poco comun; tiene un costo de una consulta por peticion.

El rate limiting se aplica por id de usuario, no por IP. Razon documentada en el codigo: la IP se puede falsear con `X-Forwarded-For` en una llamada directa al 3042. El login se limita por nombre de usuario.

El store del rate limiting es en memoria. El propio codigo advierte que alcanza para una sola instancia y que con varias replicas haria falta Redis.

Sin cache ni colas. Los envios de correo y Twilio son sincronicos dentro de la peticion: `NO DETERMINADO` si hay reintentos, no se verifico en detalle.

Componentes que NO existen

Verificado: no hay Redis, ni worker, ni cola, ni cron, ni websockets, ni aplicacion movil, ni reverse proxy propio del proyecto (el nginx que hay sirve solo los estaticos del frontend).